



Applied Cryptography

Summer 2026

Part 2: Real-World Cryptography

2.3: Secure Messaging

Nadim Kobeissi

<https://appliedcryptography.page>

Section 1

The Dark Ages

The Dark Ages

- **Pretty Good Privacy (PGP)** - Created by Phil Zimmermann in 1991
- First widely available strong encryption for everyday people
- Uses public key cryptography for email encryption and digital signatures
- Revolutionary: Previously, strong crypto was government/military only
- Core idea: Each user has a key pair (public key + private key)

How PGP works

Encryption

1. Alice writes email to Bob
2. Gets Bob's public key
3. PGP encrypts message with Bob's public key
4. Only Bob's private key can decrypt

Digital Signatures

1. Alice signs with her private key
2. Anyone with Alice's public key can verify
3. Proves message came from Alice
4. Detects tampering

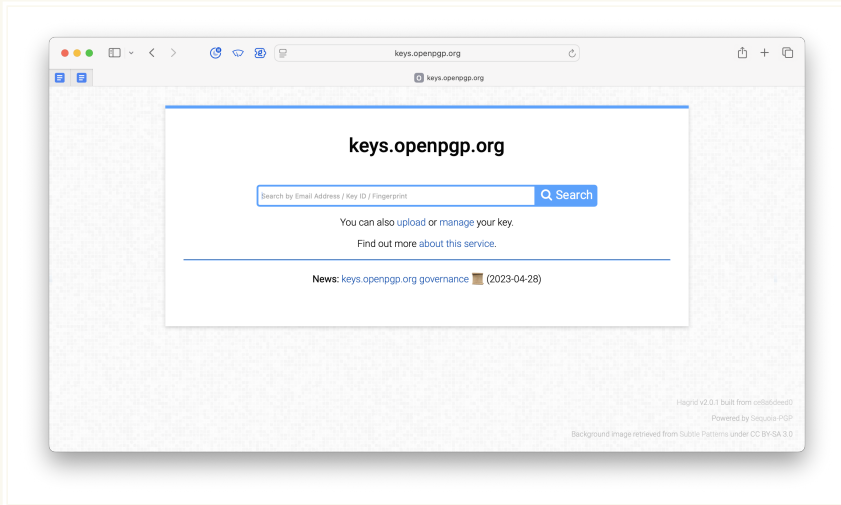
Hybrid Encryption

Actually uses symmetric crypto for message, asymmetric for key exchange

PGP in email clients

- **Early days:** Command-line tools (1991-1995)
 - Manual encryption/decryption
 - Copy and paste ciphertext into email
- **Integration era:** Plugins and extensions (1995-2010s)
 - Enigmail for Thunderbird
 - GPGTools for Apple Mail
 - Outlook plugins
- **User experience:** Still complex!
 - Key management burden on users
 - Accidental plaintext replies common
 - Attachments often forgotten

Look at this mess



A PGP “key server”

Look at this mess

```
key.pub
Plain Text

1 -----BEGIN PGP PUBLIC KEY BLOCK-----
2 Comment: 312D BB2B C13D 589B E590 E8BF BA1E 9B7A 69D4 0495
3 Comment: Nadim Kobeissi <nadin.kobeissi@outlook.com>
4 Comment: Nadim Kobeissi <nadin@cure53.de>
5 Comment: Nadim Kobeissi <nadin@nadin.computer>
6 Comment: Nadim Kobeissi <nadin@symbolic.software>
7
8 xIMEXsTf0BYJKwYBBAHwBBAQdAYenn6SPuvCoVDYSE3ha7JThbC6JvRgdok1y03
9 7AxadkPHK05hZG1t1EtYvMvpc3NpIDxuYWRpbS5rb2J1aXNzeUBvdXRab29rLnNv
10 bT7C1gQTFggAPhYhBDEtuuvBPV1Q5ZD0170em3pp1ASVBQJe5c18AhsDBQKJZg0A
11 BQsJCACCBHUKCQqLAQWAgMBAh4BAheAAAJELoem3pp1ASVgZTA+qKSWtTONVej
12 t/vfEMZQkMec+Mh+feZ85hThZgB91thvAQCyHMA8CvXnk50b2Mt233b1SMDc3G
13 116MhYMMZ2TA6cBgtNfkaM8gS291ZWJzc2kgP5shZG1LQGN1cmU1Mv5kZT7C1gQT
14 FggAPhYhBDEtuuvBPV1Q5ZD0170em3pp1ASVBQJex0CnAhsDBQKJZg6ABQsJCACc
15 BHUKCQqLAQWAgMBAh4BAheAAAJELoem3pp1ASVennAA+QFYZ2qPzCFM7KexD6yq
16 ocoBHCu/Qwt/MnAxvDorA1sSAP9s6tN1+eqwLLY4+Ht1ZPuGGE/e4c3ru44Tp0
17 TT1GBM01TnFkaWBgS291ZWJzc2kgP5shZG1LQGN1cmU1Mv5kZT7C1gQT
18 CABBANv0BQKJZg6ABQsJCACcBHUKCQqLAQWAgMBAh4BAheAFIEEM527K8E9WJ0L
19 x0LPuH0b0mUBJUFAT7E4rKCOQEACqQuP6dmmlUBJXGegp13K11au1DJC1K/Q5
20 B82ZyM1wagBP+hyU71MFs76AmhA1ccfRb125hPRB0K2Enf0wShbshyhpPiQu
21 11h0B0lwEYExYIAD4W1Q0Xk1barnT1YK0M061+6hpt6adQELQUXsTfDA1bdeLJ
22 OXYBoAULCQqLAQWAgYUcK1Cw1KEa1DAQ1eAQ1XgAAKRC6H0t6edQEL0bFAP9GGPr
23 jXKLWlncosYZ1U1750k2AYJrT16PaJ2qd05Evfd/XWpZ1dyH7D11+Kfb79e4Zer
24 xA1Zyfmj694FMBRwrnKKE5hZG1t1EtYvMvpc3NpIDxuYWRpbUwzW1b2xpYy5z
25 b2Z8d2FyZT7C1gQTFggAPhYhBDEtuuvBPV1Q5ZD0170em3pp1ASVBQJex0CnAhsDBQKJZg6ABQsJCACcBHUKCQqLAQWAgMBAh4BAheAAAJELoem3pp1ASVrmKA/01x
26 uX1p1e1jv1j1R1B6dEVNSjHNgMaZqR8u4JZD0+hoAQ3bE6VtncPzq2tNNSdL1Nv9
27 COLQPZy7Qv6Ku8x09agwBM4zBF7E3mWfCS6dAQ8B2kCPAQEHQLt81tfe+9A6nQZ
28 hqSF3Fq+L/1Nega+MlxoPK71Mbmn4EBYIACYW1QQXkLbsrnt1YK0M061+6hpt6
29 adQELQUXsTfDA1b1AUj0WYBgaAAKRC6H0t6edQEL0bFAP9GGPr
30 T3uUcok+11m0dvJH+Z+SCbsdQEA6a1d/01f0z+tlcVNMWkfvz1R+1tR/f6zd1p
31 AxDavAB00AReXN8M0go7BgEEAZdYAUQMBQdAqct1x5031VM9yYuCaAQCBgcb8
32 qB1q5TM1H+EpYhBDAQqHm4EBYIACYW1QQXkLbsrnt1YK0M061+6hpt6adQELQUX
33 sTfDA1b1AUj0WYBgaAAKRC6H0t6edQEL0bFAP9GGPrT3uUcok+11m0dvJH+Z+SCbsdQEA6a1d/01f0z+tlcVNMWkfvz1R+1tR/f6zd1p
34 /RTL1uypSR7nXu51LwEAv6n6YL83Jp5Knm7ph1spmw/nZHFYd1dKXJ4gZeuinsqL
35 =1NKS
36
37 -----END PGP PUBLIC KEY BLOCK-----
```

Allegedly my public key. Is it even? I don't know!

The Key Distribution Problem

How do you get someone's public key?

- **Key servers:** MIT, SKS keyservers
 - Upload your public key
 - Search by email/name
 - Anyone can upload anything!
- **Key fingerprints:** 40-hex-digit identifiers
 - Verify out-of-band (phone, in person)
 - Business cards with fingerprints
- **Web of Trust:** Keys signed by other users
 - "I trust Alice, Alice trusts Bob"
 - Key signing parties!
 - Such was our decadence, our confusion

Example Fingerprint

```
D745 2D8B 9E3F 4F3D 7A82
F9B5 1C4A 6E9D 8B3F 2E4C
```


PGP: fundamental challenges

- **Usability nightmare**
 - “Why Johnny Can’t Encrypt” (1999) - landmark usability study^a
 - Key management too complex for average users
 - Easy to make catastrophic mistakes
- **No forward secrecy**
 - Compromise private key = decrypt all past messages
 - Keys often used for years or decades
- **Metadata exposed**
 - Subject lines, recipients visible
 - Timing and frequency observable
- **Web of Trust failed in practice**
 - Most users never participated

^a<https://appliedcryptography.page/paper/#johnny-cant>

Why Johnny Can't Encrypt (1999)

The Study

- Whitten & Tygar tested PGP 5.0
- 12 educated email users
- 90 minutes to encrypt/sign email
- Had manual + GUI interface

Results: Catastrophic Failure

- Only 1/3 succeeded
- 1/4 sent secrets in plaintext!
- Didn't understand public keys
- Used own key to encrypt to others

Core Conclusions

- **Security $\neq$ normal software**
 - Secondary goal for users
 - Mistakes are irreversible
 - Abstract concepts
- **Good GUI isn't enough**
 - Need security-specific design
 - Must communicate mental model
- **"Usable security" requires:**
 - Users aware of security tasks
 - Can figure out how to do them
 - Don't make dangerous errors
 - Will continue using it

Why Johnny Still, Still Can't Encrypt (2015)

The Study - 16 Years Later

- Ruoti et al. tested Mailvelope
- Modern PGP browser extension
- Integrates with webmail (Gmail)
- 20 participants (10 pairs)
- Exchange encrypted email

Results: Still Catastrophic

- **Only 1/10 pairs succeeded!**
- That pair took full 45 minutes
- Success only because one knew PKI

Common Failures

- Encrypted with own public key (7/10)
- Generated keys with friend's info
- Recipients confused by PGP block
- One sent private key + password!

Pain Points

- No integrated tutorials
- PKI concepts still mystifying
- "After 5 minutes, I would have just given up and called"

Section 2

Off-the-Record Messaging: A Quick Summary

Enter OTR: Off-the-Record Messaging

- **Created in 2004** by Nikita Borisov, Ian Goldberg, and Eric Brewer
- Designed specifically for instant messaging, not email
- Key insight: Chat is synchronous, both parties online!
- **Revolutionary features:**
 - Perfect forward secrecy
 - Deniable authentication
 - Automatic key exchange

OTR's take on key exchange

PGP's Problem

- Need recipient's key *before* messaging
- Complex key distribution infrastructure
- Users must understand PKI
- One key for all messages

OTR's Solution

- Diffie-Hellman key agreement
- Keys negotiated per conversation
- Automatic and transparent
- New keys frequently (ratcheting)

OTR: new security properties

- **Forward Secrecy**
 - Each message uses ephemeral keys
 - Compromise today doesn't reveal yesterday's messages
 - Keys deleted after use
- **Deniable Authentication**
 - Messages authenticated to recipient
 - But no cryptographic proof for third parties
 - Like a real conversation - no permanent record
- **Socialist Millionaire Protocol**
 - Verify identity without trusted third party
 - Share a secret question/answer
 - Zero-knowledge proof of shared knowledge

OTR integration success

Wide Platform Adoption

- **Pidgin:** Multi-protocol IM client
- **Adium:** Mac OS X messaging
- **Kopete:** KDE instant messenger
- **Jitsi:** Voice/video/chat

Protocol Support

- XMPP/Jabber
- IRC (via proxy)
- AIM, ICQ, Yahoo Messenger
- Google Talk

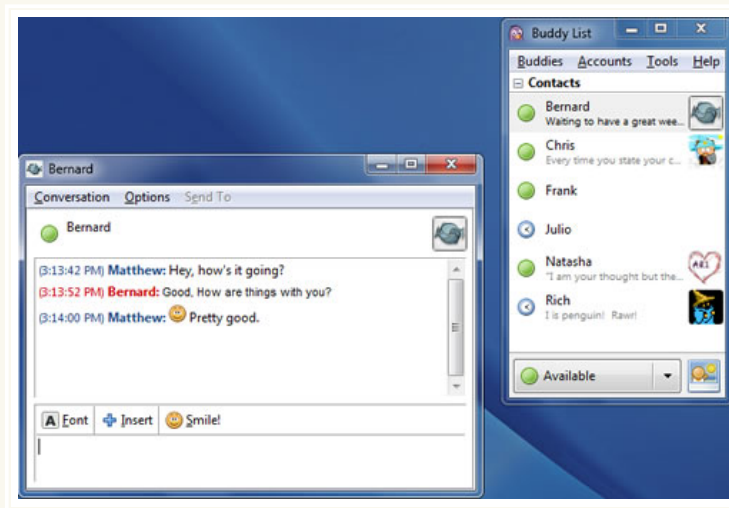
Why It Worked

- Plugin architecture
- No server changes needed
- Transparent to users
- “Just works” experience
- Graceful degradation

OTR user experience

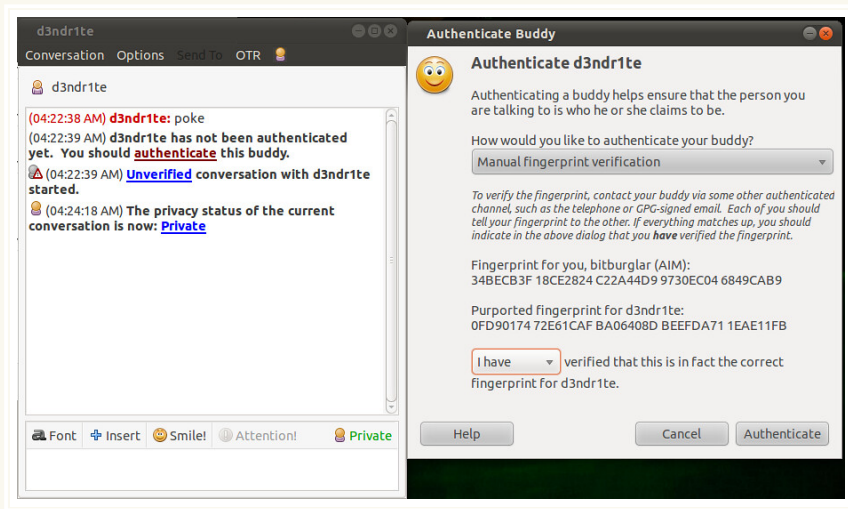
- **Minimal user intervention**
 - Click “Start private conversation”
 - OTR negotiates automatically
 - Visual indicators show encryption status
- **Optional authentication**
 - Can verify fingerprints if desired
 - Or use shared secret authentication
 - But works without it!
- **Clear security indicators**
 - “Private” vs “Not Private”
 - “Verified” vs “Unverified”
 - Simple padlock icons

Pidgin: an IM client with OTR support



Pidgin supported MSN Messenger, Yahoo Messenger, IRC, etc.

Pidgin: an IM client with OTR support



Pidgin was a popular way to use OTR.

OTR's legacy

- **Proved feasibility of usable crypto**
 - Automatic key management works
 - Users don't need to understand crypto
 - Security can be transparent
- **Established core principles**
 - Forward secrecy is essential
 - Ephemeral keys > long-term keys
 - Authentication without non-repudiation
- **Influenced modern messaging**
 - Signal Protocol builds on OTR ideas
 - WhatsApp, Signal, Wire all use similar concepts

Section 3

Diving Into Secure Messaging Cryptography through OTR

About this section

- We're going to use OTR in order to start to understand how secure messaging protocols are designed.
- A core concept we're going to expand on in some detail: **authenticated key exchange**, especially how it applies to secure messaging protocols.

OTR: new security properties

Reminder

- **Forward Secrecy**
 - Each message uses ephemeral keys
 - Compromise today doesn't reveal yesterday's messages
 - Keys deleted after use
- **Deniable Authentication**
 - Messages authenticated to recipient
 - But no cryptographic proof for third parties
 - Like a real conversation - no permanent record
- **Socialist Millionaire Protocol**
 - Verify identity without trusted third party
 - Share a secret question/answer
 - Zero-knowledge proof of shared knowledge

Regarding forward secrecy

- **What is forward secrecy?**
 - Past communications remain secure even if long-term keys are compromised
 - If attacker steals your keys today, they can't decrypt yesterday's messages
 - Critical property for long-lived messaging systems
- **How OTR achieves forward secrecy:**
 1. Uses *ephemeral* Diffie-Hellman key pairs
 2. Keys relevant only for a small number of messages in the current conversation
 3. Deleted immediately after use
 4. New keys generated frequently ("ratcheting")

Regarding forward secrecy

- Forward secrecy means that the protocol must:
 1. Introduce a notion of *ephemeral* public key pairs.
 2. Use long-term key pairs (“identity keys”) *only* in the AKE, and *only* for authentication and identity binding, not in the ratcheting steps.

Forward secrecy: key types

Long-term Identity Keys

- Persist across sessions
- Identify users to each other
- Used only for authentication
- Never directly encrypt messages
- Compromise affects identity, not past messages

Ephemeral Keys

- Generated per conversation
- Used for actual encryption
- Derived via Diffie-Hellman
- Deleted after use
- Cannot be recovered later

Key Principle

Identity keys authenticate the key exchange, but ephemeral keys encrypt the messages

Forward secrecy in practice

- **The OTR approach:**

1. Alice and Bob perform authenticated DH exchange
2. Long-term keys sign the ephemeral public values
3. Shared secret derived from ephemeral keys only
4. Message keys derived from shared secret
5. All ephemeral material deleted after use

- **Contrast with PGP:**

- PGP uses same key pair for years
- One key compromise = all messages exposed
- No automatic key rotation

SIGMA: SIGn-and-MAC Authenticated Key Exchange

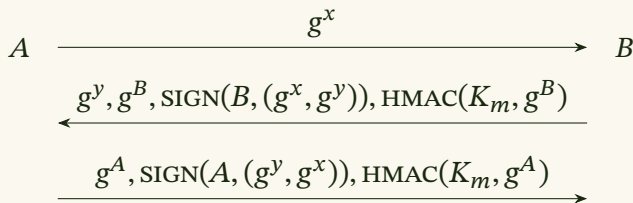
- AKE

- Both parties verify each other's identity
- Establish shared secret securely

- **Signatures for Authentication**

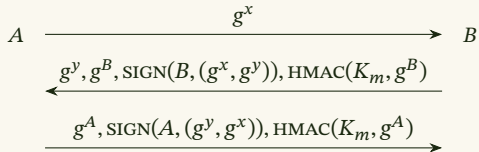
- Each party signs the DH ephemeral public keys
- Prevents man-in-the-middle attacks

- $K_m \leftarrow \text{HASH}(g^{xy})$

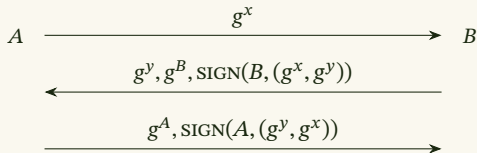


Why not simplify?

Instead of this...



...why not this?



Because...

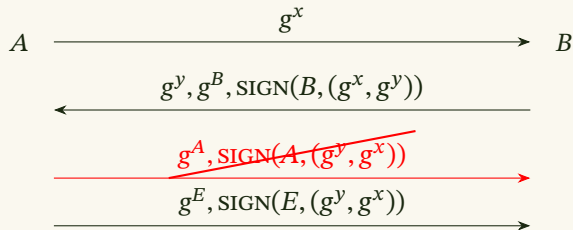
Identity substitution attack: Eve can replace Alice's identity

The Attack:

- Eve intercepts Alice's final message
- Eve sends g^E instead of g^A
- Bob accepts g^E as peer identity

Why it works:

- Signature only covers ephemeral keys
- Identity g^A not authenticated
- Eve can sign (g^y, g^x) herself

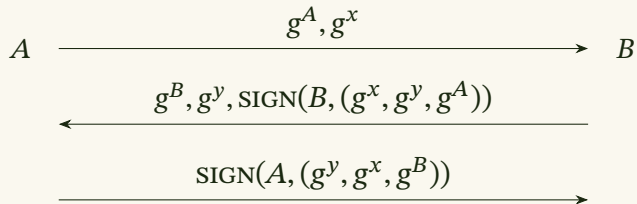


Result:

- Bob records g^E as peer
- No cryptographic link between identity and key exchange

But why HMAC though?

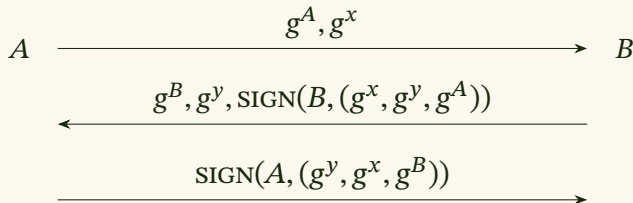
Why not just add the identity key to the signature?



But why HMAC though?

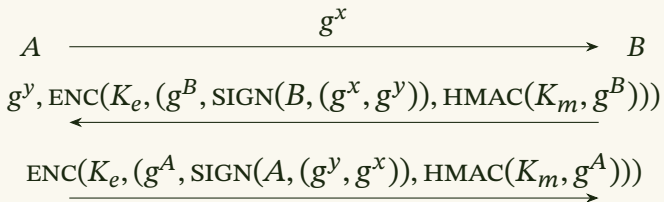
Why not just add the identity key to the signature?

- Works, but...
 - Need to know both identities before I can produce a signature,
 - Harder to implement “identity protection”
 - Identity protection?



Identity protection (variant)

Identities are encrypted and therefore cannot be learned by a passive attacker



Passive vs Active Attackers

Passive Attacker

- Eavesdropping only
- Observes network traffic
- Records all messages
- Cannot modify or inject messages
- Examples:
 - ISP logging traffic
 - Government surveillance
 - WiFi sniffing

Active Attacker

- Can modify/inject messages
- Intercepts and alters traffic
- Can impersonate parties
- Controls network routing
- Examples:
 - Man-in-the-middle attacks
 - Malicious WiFi hotspots
 - Compromised routers

Protocol Design Impact

Identity protection works against passive attackers, but active attackers can force identity revelation through protocol manipulation

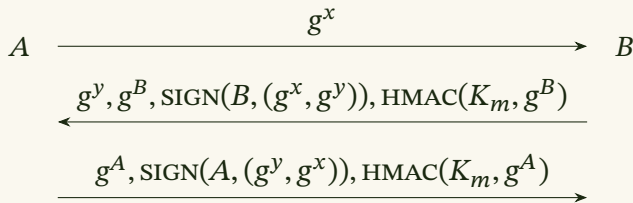
SIGMA: SIGn-and-MAC Authenticated Key Exchange

- **MACs for Identity Binding**

- MAC protects identity information
- Links identity to the key exchange
- $K_m \leftarrow \text{HASH}(g^{xy})$

- **Forward Secrecy**

- Uses ephemeral DH keys
- Past sessions safe if long-term keys compromised



Properties to consider: Identity Binding

The Problem

- How do we cryptographically tie messages to identities?
- Prevent substitution attacks
- Ensure “Bob’s key” really belongs to Bob

Identity Binding in SIGMA

- MAC includes identity: $\text{HMAC}(K_m, g^B)$
- Signature covers ephemeral keys
- Links identity $\leftrightarrow$ key exchange

Without proper binding:

- Attacker can claim others’ keys
- “Unknown Key Share” attacks
- Identity confusion attacks

Best practices:

- Include identities in authenticated data
- Sign/MAC the binding
- Verify before accepting keys

Properties to consider: Replay Attacks

What is a replay attack?

- Attacker records valid protocol messages
- Replays them later to cause confusion
- Messages are cryptographically valid!

Example scenarios:

- Replay old “I love you” after breakup
- Replay “Yes, transfer \$1000” multiple times
- Replay old key exchange messages

Defenses:

- **Nonces:** Fresh randomness each time
- **Timestamps:** Messages expire
- **Sequence numbers:** Detect duplicates
- **HKDF:** Context binding
- **AEAD:** Also a form of context binding

In secure messaging:

- OTR: Fresh ephemerals prevent replay
- Signal: Include context in authentication

Properties to consider: Key Compromise Impersonation

The Scenario:

- Alice's private key is compromised
- **Expected:** Attacker can impersonate Alice
- **KCI:** Attacker can also impersonate others *to* Alice!

Why this matters:

- Compromise should be contained
- Trust assumptions violated

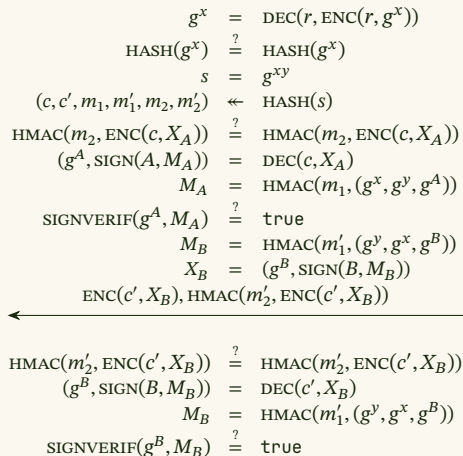
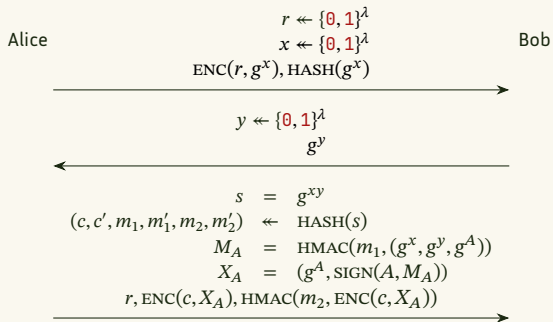
Example Attack:

- Attacker has Alice's private key
- Bob starts key exchange with Alice
- Attacker intercepts and responds as "Alice"
- But also creates fake "Bob" messages to Alice!

Preventing KCI:

- Don't use static-static DH alone
- Include ephemeral keys

OTR version 2: Authenticated Key Exchange



Section 4

Interlude: Key Derivation in Protocols

You may have noticed...

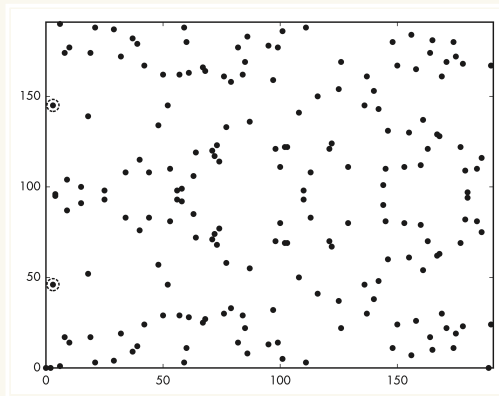
- We often use hashing to derive symmetric primitive keys from shared secrets...
 - $K_m \leftarrow \text{HASH}(g^{xy})$
 - $K_e \leftarrow \text{HASH}(g^{xy})$
 - $(c, c', m_1, m'_1, m_2, m'_2) \leftarrow \text{HASH}(s)$
 - Etc.
- This doesn't really make sense. What's *really* going on here?

We need a way to derive keys from shared secrets

- The Diffie-Hellman shared secret g^{xy} is not suitable for direct use:
 - May not be uniformly distributed
 - Could have mathematical structure that weakens security
 - We need multiple different keys for different purposes
- **Key Derivation Functions (KDFs)** solve this:
 - Extract entropy and produce uniform output
 - Can generate multiple keys from one shared secret
 - Examples: HKDF, NIST SP 800-56C, TLS-PRF
- **Proper approach:** $\text{KDF}(g^{xy}, \text{context}) \rightarrow (K_{\text{enc}}, K_{\text{mac}}, K_{\text{auth}}, \dots)$

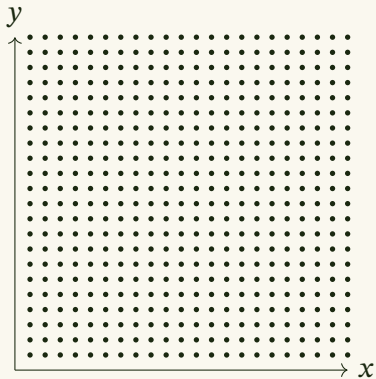
Recall the example of elliptic curves

- ECDH produces a point (or its x-coordinate) as the shared secret
- This value has mathematical structure:
 - Limited to valid curve points
 - Not all bit patterns are possible
 - Single value, but we need multiple keys
- KDFs solve this:
 $\text{KDF}(s) \rightarrow \text{uniform keys}$

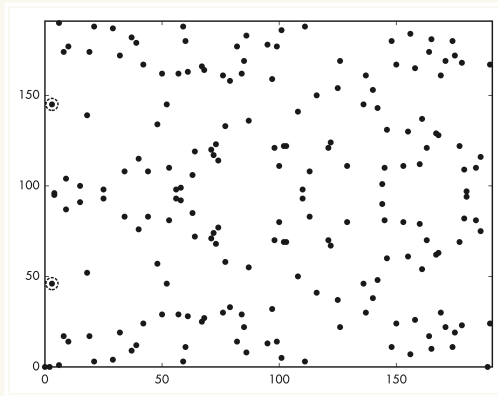


Elliptic curve over the integers. Source: Serious Cryptography

Recall the example of elliptic curves



Uniform Complete Distribution



Elliptic curve over the integers. Source: Serious Cryptography

What goes wrong without proper key derivation?

Example: Curve25519 point as AES key

- Curve25519 produces 32-byte x-coordinates
- But only $\approx 2^{252}$ valid points
- AES-256 expects uniform 256-bit keys

Concrete attack scenario:

- Attacker knows top bits are biased
- Can distinguish real keys from random
- Reduces effective key space
- Makes brute force easier

Distribution problems:

- **Bit bias:** Top 3 bits of x-coordinate never all 1
- **Modular reduction:** $x < 2^{255} - 19$
- **Invalid points:** Half of x-values have no corresponding y

Real vulnerability

In some protocols, attacker can force specific shared secrets by choosing malicious public keys from small subgroups

OTR's hand-rolled KDF

- **OTR needed multiple keys from one shared secret**
 - Encryption keys, MAC keys, counter values...
 - Needed consistent, deterministic derivation
- **Their solution (simplified):**
 - $h_0 = \text{SHA256}(s \parallel 0x00)$
 - $h_1 = \text{SHA256}(s \parallel 0x01)$
 - $h_2 = \text{SHA256}(s \parallel 0x02)$
 - Extract different keys from different hash outputs
- **Problems with this approach:**
 - No formal security analysis
 - Related-key concerns
 - Every protocol invented its own variant
 - No standard way to include context/domain separation

Enter HKDF: HMAC-based Key Derivation Function

Motivated by real needs:^a

- OTR, TLS, IPsec all needed KDFs
- Each had ad-hoc solutions
- Krawczyk (2010) formalized the problem

HKDF Design:

- **Extract:** Concentrate entropy
 - $\text{prk} = \text{HMAC}(\text{salt}, \text{ikm})$
- **Expand:** Generate keys
 - $\text{okm} = \text{HKDF-EXPAND}(\text{prk}, \text{info}, \text{len})$

Why HKDF is great:

- **Provably secure**
 - Based on HMAC security
 - Extract-then-expand paradigm
- **Domain separation**
 - “info” parameter for context
 - Different contexts → different keys
- **Flexible output**
 - Generate any length needed
 - Multiple keys from one PRK
- **Now standard:** RFC 5869

^a<https://appliedcryptography.page/paper/#hkdf-scheme>

HKDF in practice: OTR key derivation

OTR does not do this, this is an example

From one secret to many keys:

- Start with: $s = g^{xy}$
- Derive some unique session ID as the salt
- `HKDF(ikm, info, salt, len)`

OTR Key Derivation

```
c = HKDF(s, "OTR-ENC-Alice", sessionID, 32)
c' = HKDF(s, "OTR-ENC-Bob", sessionID, 32)
m1 = HKDF(s, "OTR-MAC1-Alice", sessionID, 32)
m1' = HKDF(s, "OTR-MAC1-Bob", sessionID, 32)
m2 = HKDF(s, "OTR-MAC2-Alice", sessionID, 32)
m2' = HKDF(s, "OTR-MAC2-Bob", sessionID, 32)
```

Domain separation benefits:

- **Clear purpose:** Each key has explicit context
- **No key reuse:** Different contexts = different keys
- **Direction-specific:** Alice/Bob get different keys

Compare to ad-hoc approach:

- No semantic meaning
- Easy to mess up counters
- No formal analysis

Section 5

Back to OTR: Sending and Receiving Messages

OTR: message exchange and ratcheting

- **Each message includes:**
 - The encrypted message itself
 - A new ephemeral DH public key share
 - MAC authenticating the entire message
- **On sending/receiving a message:**
 1. Generate new ephemeral key pair: $x_{\text{new}} \leftarrow \{0, 1\}^\lambda$
 2. Include $g^{x_{\text{new}}}$ with the message
 3. Compute new shared secret: $s_{\text{new}} = g^{x_{\text{new}} \cdot y_{\text{current}}}$
 4. Derive new keys from s_{new}
 5. Delete old ephemeral private key
- **MAC key revelation (for deniability):**
 - Once both parties have acknowledged receipt
 - Publish the old MAC keys in cleartext
 - Anyone could now forge past messages!
 - No cryptographic proof of who sent what

Forward secrecy example: stolen laptop

The Scenario:

- You're having an OTR conversation
- 30 messages already exchanged
- Your laptop gets stolen!
- Attacker has:
 - Your current ephemeral key
 - Your long-term identity key
 - The OTR client state

What the attacker CAN do:

- Continue the conversation as you
- Read messages buffered in memory
- Decrypt future messages (until detected)

What the attacker CANNOT do:

- Decrypt your past 30 messages
- Recover deleted ephemeral keys
- Prove you sent past messages

Limitations to forward secrecy in practice

Key erasure challenges:

- **Memory persistence**
 - Keys may remain in RAM
 - Swap files on disk
 - Hibernation files
 - Core dumps
- **Language/OS limitations**
 - No guaranteed secure erasure
 - Garbage collection delays
 - Memory optimization copies

Synchronization vs Security:

- **The dilemma:**
 - Messages arrive out of order
 - Network delays/retries
 - Must decrypt when received
- **Signal/WhatsApp solution:**
 - Keep window of old keys
 - “Message key cache”
 - Trade-off: usability vs security
- **Real impact:**
 - Forward secrecy weakened
 - Attack window extended
 - But better UX

Deniability as a security property

What is deniability?

- Messages authenticated to recipient
- But no cryptographic proof for third parties
- **After conversation:** anyone could have created the transcript

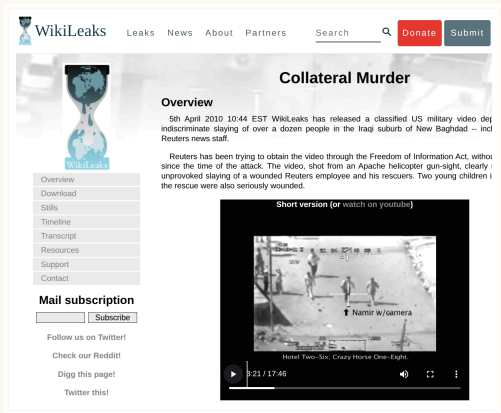
OTR's approach:

- **MACs instead of signatures**
 - Both parties have MAC key
 - Either could forge messages
 - Old MAC keys revealed
 - Now anyone can forge past messages

Contrast with digital signatures:

- **PGP-signed email:**
 - Permanent proof of authorship
 - Third party can verify later
 - Non-repudiation by design
- **OTR philosophy:**
 - Like face-to-face conversation
 - No permanent cryptographic record
 - Protects against coercion

The deniability debate: Manning/Lamo case



WikiLeaks Leaks News About Partners Search

Collateral Murder

Overview

5th April 2010 10:44 EST WikiLeaks has released a classified US military video depicting indiscriminate slaying of over a dozen people in the Iraqi suburb of New Baghdad -- incl Reuters news staff.

Reuters has been trying to obtain the video through the Freedom of Information Act, without success since the time of the attack. The video, shot from an Apache helicopter gun-sight, clearly depicts the unprovoked slaying of a wounded Reuters employee and his rescuers. Two young children in the rescue were also seriously wounded.

Short version (or watch on youtube)

Hotel Two-Six, Crazy Horse One-Eight.

8:21 / 17:46

Follow us on Twitter!
Check our Reddit!
Digg this page!
Twitter this!

Collateral Murder, WikiLeaks Page

The Collateral Murder Video:

- Classified U.S. military footage from July 2007
- Apache helicopter attack in Baghdad
- Killed 11 people including 2 Reuters journalists
- Allegedly leaked by Chelsea Manning to WikiLeaks
- Published April 2010

The deniability debate: Manning/Lamo case

The 2010 incident:

- Chelsea Manning allegedly used OTR with Adrian Lamo
- Allegedly discussed classified document leaks
- Lamo saved chat logs, gave to FBI
- **Used as evidence in court**

Why deniability failed:

- Courts don't require cryptographic proof
- Context and corroboration matter more
- One party (Lamo) testified to authenticity
- Technical deniability $\neq$ legal deniability

The deniability debate: Is deniability worth anything?

Is deniability worth anything?

- **Arguments for:**
 - Prevents automatic mass verification
 - Raises doubt about leaked transcripts
 - Protects against future proof requirements
- **Arguments against:**
 - Courts accept weaker evidence
 - Screenshots can be just as damning
 - Adds protocol complexity
 - Users don't understand it

Open question

Does cryptographic deniability provide meaningful real-world protection?

Attacks on OTR version 2

Version Rollback Attack^a

- Version negotiation happens before authentication
- Attacker can force use of older, weaker version
- **Attack scenario:**
 - Attacker modifies messages to claim only OTRv1
- **The fix:**
 - Include version number in AKE
 - Authenticate the negotiated version

^a<https://appliedcryptography.page/paper/#otr-analysis>

Authentication Failure

- Mallory can convince Alice she completed AKE with Bob
- Bob has no knowledge of this exchange
- Only affects AKE completion belief
- **Important limitations:**
 - No actual MITM possible
 - Cannot decrypt messages
 - Cannot impersonate either party
- **Real impact:**
 - Alice thinks she's secure with Bob
 - Sends encrypted messages Bob can't read

OTRv2 message integrity attack

Attack Setup:

- Exploits MAC key revelation + message ordering
- Requires network control (active attacker)

Key Insight:

- Alice reveals MAC keys after “finishing” with them
- Assumes messages arrive in order, but attacker can delay/reorder messages!
- Combined with malleable encryption = attack

Attack Prerequisites:

- **MAC key revelation** for deniability
- **Malleable encryption** (stream cipher)
- **Network control** by attacker
- **Message ordering assumptions**

Message integrity attack: why it works

Step-by-step:

1. Alice sends m_0 with keys (x_0, y_0)
2. Bob responds with m_1 using (x_1, y_0)
3. Alice sees Bob moved to x_1 , assumes (x_0, y_0) done
4. Alice publishes MAC keys for (x_0, y_0)
5. Mallory blocks Alice's next message
6. Mallory creates forged m'_0 using:
 - Published MAC keys
 - Malleable encryption
7. Bob accepts as delayed message

Why Bob accepts it:

- Message appears to precede m_1
- Valid MAC (Mallory has the key!)
- Network delays are common
- No way to detect forgery

Attack Impact:

- Insert forged messages
- Modify past messages
- Break conversation integrity
- Deniability feature enables attack!

Fixing the message integrity attack

Potential solutions:

- **Delayed MAC revelation**
 - Wait longer before publishing
 - Ensure truly done with keys
- **Better state machine**
 - Track message expectations
 - Reject out-of-order messages

Additional improvements:

- **Use authenticated encryption**
 - Not just MAC + malleable cipher
 - Modern AEAD constructions
- **Include counters in MAC**
 - Prevent message reordering
 - Detect replay attempts

Section 6

Signal

From synchronous to asynchronous messaging

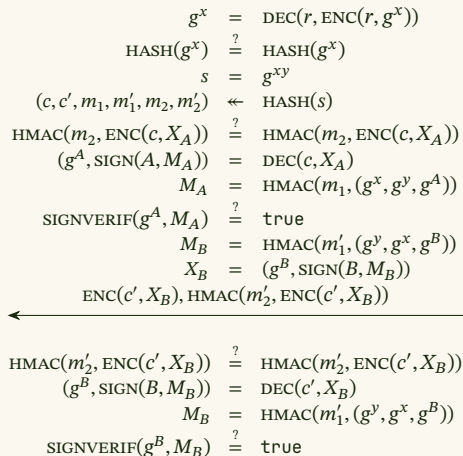
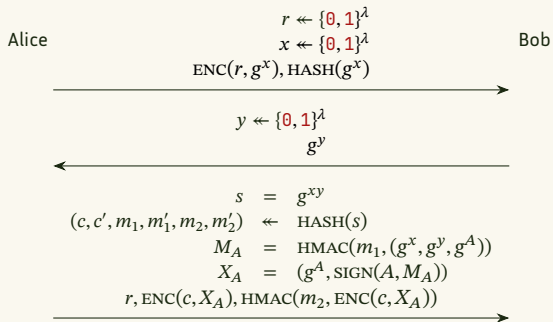
OTR: Synchronous Protocol

- Both parties must be online
- Real-time key exchange required
- Can't send to offline users
- **User experience:**
 - "Bob is offline"
 - Wait for Bob to come online
 - Then initiate secure chat
- Works for MSN Messenger and the early 2000s
- Not suitable for mobile era

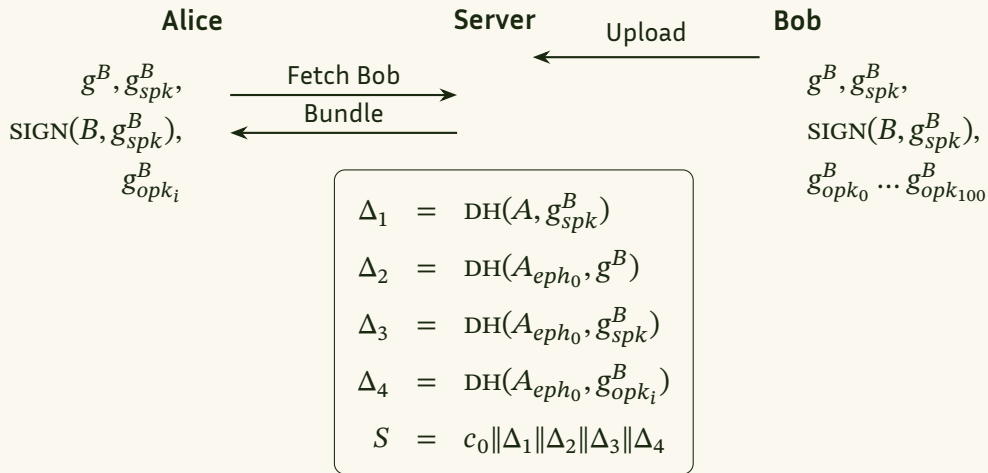
Signal: Asynchronous Protocol

- Send messages anytime
- Recipient can be offline
- **SMS-like experience:**
 - Send message to Bob's phone
 - Bob's phone is off
 - Bob receives when phone turns on
- **Challenge:** How to do AKE without Bob?
- **Solution:** Pre-keys on server

OTR version 2: Authenticated Key Exchange



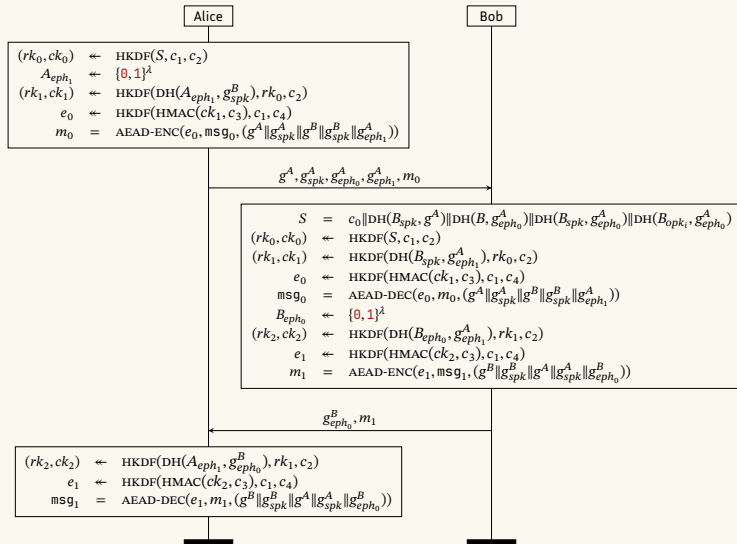
X3DH: Signal's Asynchronous Key Exchange



X3DH: Signal's Asynchronous Key Exchange

- *spk*: "Signed pre-key": a signed ephemeral key
 - Replaced every 2 days
- *opk*: "One-time" ephemeral pre-key
 - Bob uploads 100 of them, used only once
- *eph*: Ephemeral keys, generated during the session
- $c_0 \dots c_n$: Various constants found in the protocol
- Alice can obtain S even if Bob is offline!
- Now, using S , Alice can derive a bunch of keys to get the session going

Signal Double Ratchet



Why the Double Ratchet is nice

- **Every message gets fresh keys**
 - Chain key advances: $ck_{i+1} = \text{HMAC}(ck_i, c_5)$
 - Message key derived: $e_i = \text{HKDF}(\text{HMAC}(ck_i, c_3), c_1, c_4)$
 - Old chain keys immediately deleted
 - Compromise of e_i reveals nothing about e_{i-1} or e_{i+1}
- **Two types of ratcheting**
 - **Symmetric ratchet:** HMAC chain for immediate messages
 - **DH ratchet:** New ephemeral keys when direction changes
 - Root key combines both: $(rk_{new}, ck_{new}) = \text{HKDF}(\text{DH}(\text{new ephemerals}), rk_{old}, c_2)$
- **Post-compromise security!**
 - If attacker steals your state at time t ...
 - After one DH ratchet step, they're locked out again
 - Fresh randomness from new ephemeral keys
 - OTR couldn't do this - required interactive AKE

Forward Secrecy vs Post-Compromise Security

Forward Secrecy

- **Past messages stay secret**
- If keys compromised today, yesterday's messages remain encrypted
- “Looking backward” protection
- **How it works:**
 - Delete old keys after use
 - Can't decrypt without keys
 - Time travel won't help attacker
- **Example:** Laptop stolen after conversation - past messages safe

Post-Compromise Security

- **Future messages become secret again**
- If keys compromised today, tomorrow's messages will be secure
- “Healing” or “self-healing” property
- **How it works:**
 - Inject fresh randomness
 - New ephemeral keys
 - Attacker locked out again
- **Example:** Laptop stolen but recovered - future messages re-secure

Signal's Sealed Sender: the metadata problem

What E2E encryption doesn't hide:

- Message contents are protected
- But service still sees who talks to whom
- Traditional messaging shows:
 - Sender identity (authentication)
 - Recipient identity (routing)

Why authentication was needed:

- Prevent spoofing
- Enable rate limiting
- Block abuse
- Trace bad actors

The challenge:

- How to hide sender from service
- While preventing abuse?

Signal's Sealed Sender: the solution

Sender Certificates

- Short-lived, signed by service
- Contains:
 - Phone number
 - Identity key
 - Expiry timestamp
- Included inside encrypted envelope from Alice to Bob
- Service can see it at issuance, but not while it's inside encrypted envelope.

Delivery Tokens

- 96-bit token derived from profile key
- Required to send sealed messages
- Shared via existing profile system
- Acts as anti-abuse mechanism

Trade-off:

- Default: Only contacts can use sealed sender
- Optional: Accept from anyone (more spam risk)

Sealed Sender: technical details

Double Encryption Approach:

1. Message encrypted with Signal Protocol as usual
2. Envelope (sender cert + ciphertext) encrypted to recipient's identity key
3. Service sees only encrypted blob + delivery token

Envelope Encryption (simplified):

- Generate ephemeral key pair: (e_{pub}, e_{priv})
- ECDH with recipient's identity key
- Derive encryption keys via HKDF
- Encrypt sender identity + certificate
- Second layer using sender's identity key for authentication

Result

Service can route messages without knowing sender identity

Sealed Sender: does it really work?

- **The Attack (2021): Statistical Disclosure^a**

- Sealed sender hides who sent the message
- But delivery receipts create a timing pattern
- When Bob receives a message, he often replies quickly
- This creates a “reply epoch” - a time window

- **The Clever Observation:**

- Alice sends to Bob → Bob likely replies within epoch
- Other users also send/receive during this time
- But Alice appears more often in Bob’s reply epochs

- **The Attack Algorithm:**

1. Monitor when Bob receives messages
2. Count who receives messages right after (target epochs)
3. Compare to random time windows
4. Alice’s count grows, others stay near zero
5. After just a few messages: Bob’s contacts revealed!

^a<https://appliedcryptography.page/paper/#sealed-sender>

Can your message truly be secure after a hack?

The Promise and Reality of Post-Compromise Security^a

- Based on: “Impossibility Results for Post-Compromise Security” by Cremers, Medinger, and Naska (2024)
- **Spoiler:** The answer might surprise you...

^a<https://appliedcryptography.page/paper/#pcs-impossibility>

The Dream: Self-Healing Security

The Promise of PCS:

- Your phone gets hacked today
- Attacker reads all your secrets
- But tomorrow...
- **Magic happens!**
- Future messages become secure again
- Attacker is “locked out”

How it “works”:

- Continuously inject fresh randomness
- Update keys with every message
- If attacker misses an update...
- They can’t decrypt future messages!
- Like changing locks after a break-in

Question for the class

Does this sound too good to be true?

PCS reality check

Academic papers say:

- “Signal Protocol achieves PCS!”
- “Double Ratchet provides healing!”
- “Proven secure!”
- Lots of formal proofs! Wow!

This paper says:

- Wait, not so fast...
- Those proofs assume perfect conditions
- Real apps $\neq$ Academic protocols
- **Users don't actually get PCS!**

Phones live in the real world

Things that happen to real phones:

- **Hardware failures:** Bit flips, memory corruption
- **Software issues:** App crashes, OS updates
- **User actions:** Restore from backup, reinstall apps
- **Physical events:** Drop phone in toilet

What happens to your chats when these occur?

- Academic answer: “You’re locked out forever!”
- Real-world requirement: “I need to keep chatting!”
- If WhatsApp doesn’t meet that requirement, it will lose users

Sessions all the way down

What users see:

- One conversation with Bob
- Linear list of messages
- Simple and clean

What's really happening:

- Multiple concurrent “sessions”
- Signal allows up to **40 sessions!**
 - WhatsApp even more, I believe
- Each can be compromised separately
- Attacker can start new ones

The Impossibility Result

If your app must handle state loss (all real apps do), then full conversation-level PCS is **mathematically impossible**

Why 40 sessions?

Signal's Magic Number: 40

- Not arbitrary - represents a design choice
- Each session = tolerance for one “desync event”
- More sessions = Better usability, Worse security

N = 1 (Ultra secure)

- Minimal attack surface
- Constant “can’t decrypt” errors
- Messages lost during network delays

N = 40 (Signal's choice)

- Handles many failure scenarios
- Good user experience
- 40 ways for attacker to get in!

Using Tamarin to crush dreams

How they proved the impossibility:

1. Model abstract communication system in Tamarin prover
2. Add real-world requirements:
 - Must handle dynamic state loss (crashes, backups)
 - Must handle static state loss (lost devices)
 - Must allow recovery to continue chatting
3. Try to prove PCS property holds...
4. **Tamarin finds counterexamples!**

So what can we do?

If we can't have perfect PCS, can we do better?

- **Reduce session count:** Maybe $N = 2$ instead of 40?
- **Sequential sessions:** Old sessions can only receive, not send
- **Time limits:** Delete old sessions after time T
- **UI warnings:** Highlight messages from old sessions

But remember:

- These are band-aids, not cures
- Fundamental trade-off remains
- Perfect security = Unusable app

Discussion time

1. Is post-compromise security just security theater?

- If users don't actually get it, why do we pretend?
- Marketing vs. reality?

2. What would you choose?

- Ultra-secure but breaks constantly?
- Works smoothly but weaker guarantees?

3. Can we do better?

- Hardware security modules?
- Out-of-band verification?
- Just give up on state loss resilience?

4. Who's being honest with users?

- Should apps disclose these limitations?
- Do users even care?



Academia versus the real world

Section 7

Telegram

What about Telegram?

Telegram's Approach

- Two modes: Cloud chats vs Secret chats
- Cloud chats: Encrypted to server, not E2E
- Secret chats: Custom MTProto 2.0 protocol
- Only available in 1-on-1 conversations
- Must be explicitly enabled

Key Differences from Signal

- Most users never use secret chats
- Custom crypto instead of proven protocols
- Server can read most messages
- Focus on features over security
- Different threat model assumptions

Security Reality

Despite marketing, most Telegram usage is **not** end-to-end encrypted

Telegram's MTProto

What Telegram Did

- Designed custom encryption protocol
- Ignored existing academic work
- No formal security analysis initially
- Multiple versions after vulnerabilities found

The Cryptographic Community Response

- "Don't roll your own crypto!"
- Multiple published attacks
- Academic papers showing weaknesses
- Telegram's defensive responses

Why Telegram made these choices

- **Server-side features require plaintext access:**
 - Cloud sync across unlimited devices
 - Powerful search across all messages
 - Large file sharing and streaming
 - Bots and integrations
 - Message editing and deletion
- **Different threat model:**
 - “We don’t trust governments, but we trust ourselves”
 - Emphasis on government censorship resistance
 - Less focus on protecting from service provider
- **User experience priorities:**
 - Instant sync, no waiting for key exchange
 - Works the same across all platforms
 - No “this message couldn’t be decrypted” errors

The Telegram Paradox

What activists think they're getting:

- End-to-end encrypted messaging
- Protection from government surveillance
- Secure communication platform
- "Telegram is encrypted, right?"

What they're actually getting:

- Messages readable by Telegram servers
- Subject to legal requests and seizure
- Only secret chats are E2E encrypted
- Most users never enable secret chats

Real-world Impact

Pavel Durov can read your Telegram chats at his discretion

Section 8

Group Secure Messaging

The Group Messaging Problem

Two-party protocols work great for...two parties

- Signal Protocol: Alice ↔ Bob
- OTR: Real-time 1-on-1 chat
- X3DH: Asynchronous key agreement
- Double Ratchet: Forward secrecy

But what about groups?

- Family group chat (5 people)
- Work team (20 people)
- Community groups (100+ people)

Security challenges:

- Members join and leave
- Forward secrecy for leavers
- Post-compromise security
- Scalability requirements

How Signal/WhatsApp handle groups today

The Pairwise Channel Approach

- No true group protocol
- Sender encrypts to each member
- For n members: $n - 1$ encryptions
- Each member has separate ratchet

Problems with this approach:

- **Linear scaling:** $\mathcal{O}(n)$ work per message
- **Bandwidth:** Upload grows with group size
- **Battery drain:** More crypto operations
- **Consistency:** Members may see different states

WhatsApp's approach: sender keys

How Sender Keys Work:

- Each group member has a “sender key”
- Shared with all other members
- One encryption per message (not per recipient!)

Sender Key Components:

- $SK = (spk, ck)$
- spk : Public signature key
- ck : Symmetric chain key
- Chain key ratchets forward

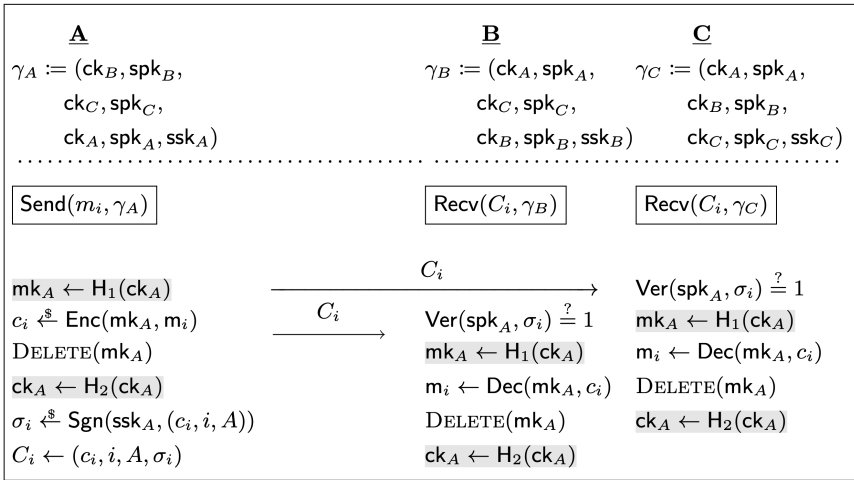
Sending a Message:

1. Derive message key: $mk = H_1(ck)$
2. Encrypt: $c = \text{ENC}(mk, m)$
3. Sign: $\sigma = \text{SIGN}(ssk, c)$
4. Erase mk immediately
5. Ratchet: $ck_{new} = H_2(ck)$

Benefits:

- $\mathcal{O}(1)$ encryptions per message
- Handles out-of-order delivery
- Scales to large groups

WhatsApp's approach: sender keys



Source: David Balbás, Daniel Collins and Phillip Gajland, *WhatsApp with Sender Keys? Analysis, Improvements and Security Proofs*, IACR Asiacrypt, 2023.

Sender keys: trade-offs

What we gain:

- **Efficiency:** Single encryption
- **Scalability:** Works for 256+ members^a
- **Battery life:** Less crypto work
- **Bandwidth:** Constant message size

^aRecently increased to 1,024.

What we lose:

- Weaker forward secrecy
- Weaker post-compromise security
- Malicious server can add/remove parties

Enter MLS: Messaging Layer Security

IETF Standard (RFC 9420, July 2023)

- Designed for efficient group messaging
- Single encryption per message
- Logarithmic scaling: $\mathcal{O}(\log n)$
- Native group key agreement
- **Group key agreement:** All members share same key
- **Ratcheting epochs:** Keys evolve over time
- **Forward secrecy:** Past epochs stay secure
- **Post-compromise security:** Healing after breach

Quick note: HPKE

Hybrid Public Key Encryption (RFC 9180)^a

- Combines asymmetric + symmetric crypto
- Encrypts to public key, no interaction needed
- Used in MLS, TLS Encrypted Client Hello, and more

Two-step process:

1. **Encapsulation:** Generate shared secret
2. **Seal:** Encrypt data with that secret

^a<https://www.rfc-editor.org/rfc/rfc9180.html>

Simple Example:

Sender (Alice)

```
// Bob's public key: pk_bob
(enc, ctx) = HPKE.Setup(pk_bob)
ciphertext = ctx.Seal("Hello Bob!")
// Send: (enc, ciphertext)
```

Receiver (Bob)

```
// Bob's private key: sk_bob
ctx = HPKE.Setup(enc, sk_bob)
plaintext = ctx.Open(ciphertext)
// plaintext = "Hello Bob!"
```

Key benefit: One-shot encryption without prior key exchange!

TreeKEM: use a tree to manage group AKE

Tree of Subgroups:

- Each node = subgroup with secret (e.g., s_{abc})
- Corresponding public key (e.g., pk_{abc})
- Example: s_{abcde} is the group key

Member Knowledge:

- Member b knows: $s_{ab}, s_{abc}, s_{abcde}$
- Only secrets on path to root
- Cannot compute sibling secrets

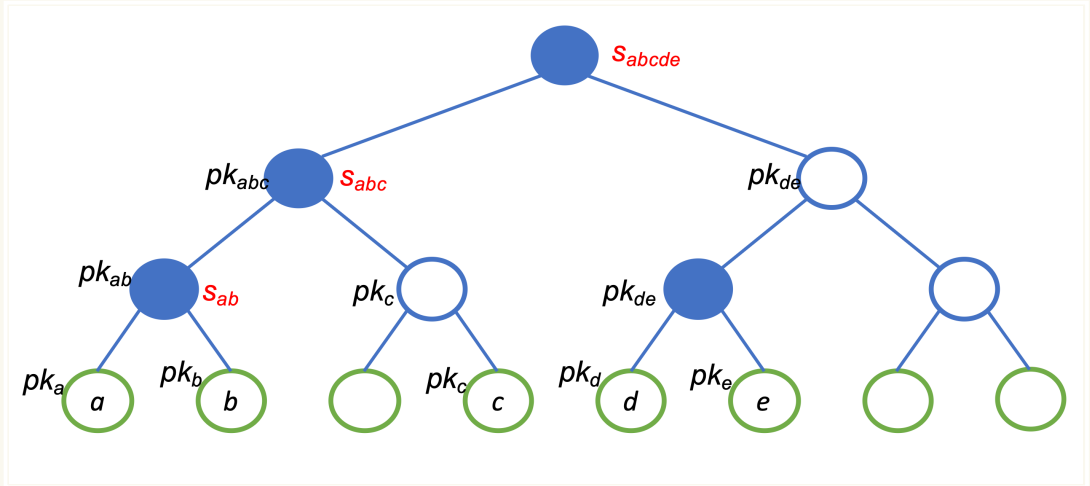
Updating Keys (Commit):

- Member b updates its path:
 - $s_{ab} \rightarrow s'_{ab}$
 - $s_{abc} \rightarrow s'_{abc}$
 - $s_{abcde} \rightarrow s'_{abcde}$
- Encrypt to siblings:
 - $\text{HPKE}(pk_c, s'_{abc})$
 - $\text{HPKE}(pk_{de}, s'_{abcde})$

Efficiency Win

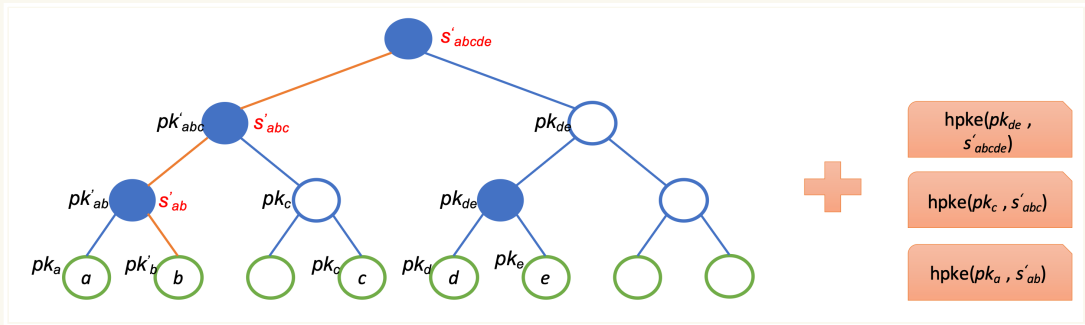
For n members: Only $\log(n)$ encryptions!
Example: 8 members = 3 encryptions

TreeKEM



Source: Théophile Wallez

TreeKEM



Source: Théophile Wallez

MLS: reality check

The Complexity Problem:

- **Massive specification:** RFC 9420 is 132 pages!
- **Implementation nightmare:**
 - Multiple tree operations
 - Complex state management
 - Intricate error handling
- **Correctness is hard:**
 - Easy to get wrong
 - Subtle security bugs
 - Few complete implementations

Developer Hostility:

- **No standard API:**
 - Each implementation different
 - No drop-in replacement
 - Steep learning curve
- **Infrastructure requirements:**
 - Need custom delivery service
 - Complex server-side logic
 - State synchronization issues



Applied Cryptography

Summer 2026

Part 2: Real-World Cryptography

2.3: Secure Messaging

Nadim Kobeissi

<https://appliedcryptography.page>